LESSON 02 · GIT · 50 MIN

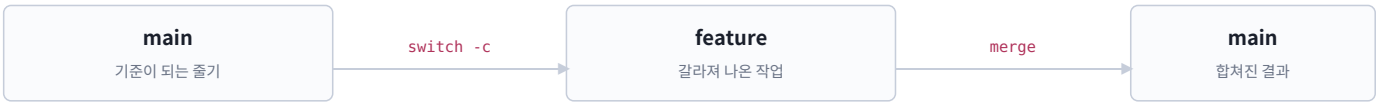
2교시 브랜치와 병합 — 갈래를 나누고 합치기

fast-forward와 3-way 병합은 내가 고르는 옵션이 아니라, 커밋을 어디에 쌓았는지에 따라 결정되는 결과입니다. 두 경우를 직접 만들어 보고, 충돌을 일부러 내서 해결까지 해 봅니다.

개념 12분 · 실습 28분 · 정리와 질문 10분

OVERVIEW

브랜치는 커밋을 가리키는 이름표



> 브랜치를 만들어도 커밋은 하나도 늘지 않습니다. 이름표가 하나 더 생길 뿐입니다. **HEAD** 는 '내가 지금 보고 있는 곳'을 뜻하고, 커밋은 항상 HEAD가 가리키는 브랜치에 붙습니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`git branch`	브랜치 목록 확인 (* 가 현재 위치)	지금 어디에 있는지 볼 때
`git switch -c <이름>`	브랜치를 만들고 그리로 이동	새 기능을 시작할 때
`git switch <이름>`	브랜치 이동	작업을 전환할 때
`git merge <이름>`	현재 브랜치에 상대 브랜치를 합침	기능이 완성됐을 때
`git log --oneline --graph --all`	그래프로 브랜치 모양 보기	헷갈릴 때 가장 먼저
`git branch -d <이름>`	병합이 끝난 브랜치 삭제	작업 정리
`git merge --abort`	병합을 취소하고 이전 상태로	충돌이 감당 안 될 때
`git stash / git stash pop`	작업을 잠시 치워두기 / 되돌리기	커밋하기 애매한데 이동해야 할 때

HANDS-ON

실습 순서

STEP 01 현재 상태 확인

1교시 폴더를 그대로 사용합니다.

```
cd git-practice
git log --oneline
git branch
git status
```

확인 포인트

- [ ] 커밋이 2개 이상 있다
- [ ] 브랜치가 main 하나이고 * 가 붙어 있다
- [ ] working tree clean 상태다

> 브랜치를 나누기 전에는 반드시 **working tree clean** 이어야 합니다. 커밋 안 한 수정이 있으면 이동할 때 문제가 생깁니다.

STEP 02 브랜치 만들고 이동하기

브랜치 생성이 커밋을 늘리지 않는다는 것을 확인합니다.

```
git log --oneline | wc -l
git switch -c feature-login
git branch
git log --oneline | wc -l
```

확인 포인트

- [ ] * 표시가 feature-login 으로 옮겨졌다
- [ ] 커밋 개수가 전혀 늘지 않았다

> 윈도우 CMD에는 wc 가 없습니다. 그냥 **git log --oneline** 을 두 번 보고 줄 수를 눈으로 비교하세요.

STEP 03 feature 브랜치에서 작업하기

커밋이 현재 브랜치에만 쌓이는 것을 확인합니다.

```
echo "로그인 페이지" > login.txt
git add login.txt
git commit -m "로그인 페이지 추가"
git log --oneline --graph --all
```

확인 포인트

- [ ] feature-login 이 새 커밋을 가리키고, main 은 뒤에 남아 있다

STEP 04 fast-forward 병합

main이 뒤처져 있기만 할 때 어떤 일이 일어나는지 봅니다.

```
git switch main
ls
git merge feature-login
ls
git log --oneline --graph
```

확인 포인트

- [ ] switch 직후 ls 에 login.txt 가 없다 — 브랜치를 옮기면 파일도 바뀐다
- [ ] merge 결과에 Fast-forward 라는 단어가 보인다
- [ ] 병합 커밋이 새로 생기지 않았고 그래프가 여전히 일직선이다

> main이 feature의 조상이기만 하면 Git은 새 커밋을 만들지 않고 **이름표만 앞으로 옮깁니다.**

STEP 05 끝난 브랜치 정리

병합된 브랜치를 지웁니다.

```
git branch -d feature-login
git branch
```

확인 포인트

- [ ] feature-login 이 목록에서 사라졌다
- [ ] login.txt 는 그대로 남아 있다 (내용은 이미 main에 합쳐졌으므로)

STEP 06 3-way 병합 만들기

양쪽이 각자 나아간 상황을 만들어 병합 커밋을 생성합니다.

```
git switch -c feature-b
echo "B 기능" > b.txt && git add . && git commit -m "B 추가"
git switch main
echo "A 기능" > a.txt && git add . && git commit -m "A 추가"
git merge feature-b
git log --oneline --graph --all
```

확인 포인트

- [ ] 에디터가 열리면 기본 메시지 그대로 저장하고 닫는다 (vim이면 :wq 입력)
- [ ] Merge branch 'feature-b' 라는 커밋이 새로 생겼다
- [ ] 그래프에서 선이 갈라졌다가 다시 합쳐진다

> 이 병합 커밋은 **부모가 둘**입니다. 이번 병합처럼 서로 다른 파일을 건드렸다면 충돌은 나지 않습니다.

STEP 07 충돌 만들고 해결하기 — 오늘의 핵심

같은 파일 같은 줄을 양쪽에서 고쳐 충돌을 재현합니다.

```
git switch -c fix-title
echo "제목: 우리 서비스" > title.txt && git add . && git commit -m "한글 제목"
git switch main
echo "제목: Our Service" > title.txt && git add . && git commit -m "영문 제목"
git merge fix-title
git status
# title.txt 를 편집기로 열어 원하는 내용만 남기고 표시자를 지웁니다
git add title.txt
git commit
```

확인 포인트

- ☐ CONFLICT (add/add): Merge conflict in title.txt 메시지가 나온다
- ☐ status가 Unmerged paths 라고 알려준다
- ☐ 파일 안에 <<<<<<< HEAD, =======, >>>>>>> fix-title 세 줄이 들어 있다
- ☐ 표시자를 지우고 add + commit 하면 status가 clean 으로 돌아온다

> 위쪽이 **내가 있던 브랜치(HEAD)**, 아래쪽이 **가져온 브랜치**입니다. 충돌 해결은 둘 중 하나를 고르는 일이 아니라 올바른 코드를 만드는 일입니다. 감당이 안 되면 **git merge --abort** 로 병합 직전 상태로 돌아갈 수 있습니다.

STEP 08 stash 로 작업 잠시 치워두기

커밋하기 애매한 작업을 두고 브랜치를 옮기는 방법을 익힙니다.

```
echo "쓰다 만 작업" >> a.txt
git status
git stash
git status
git stash list
git switch feature-b
git switch main
git stash pop
git status
```

확인 포인트

- ☐ stash 직후 working tree clean 이 된다
- ☐ stash list 에 stash@{0} 이 보인다
- ☐ pop 하면 치워둔 수정이 그대로 돌아온다

> stash는 임시 보관함입니다. 오래 쌓아두면 무엇을 넣었는지 잊게 되니, 하루를 넘길 작업이면 임시 커밋을 만드는 편이 낫습니다.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
`CONFLICT (content): Merge conflict in ...`	같은 줄을 양쪽에서 수정	파일 열어 표시자 삭제 → git add → git commit · 또는 git merge --abort
`Your local changes would be overwritten`	커밋 안 한 수정이 있는 채로 이동	git commit 하거나 git stash 로 치운 뒤 이동
`The branch 'x' is not fully merged`	병합 안 된 브랜치를 삭제 시도	내용을 정말 버릴 거면 git branch -D x (대문자 D)
병합 중 에디터가 열리고 못 빠져나옴	기본 에디터가 vim	Esc → :wq → Enter · 또는 git config --global core.editor "code --wait"
`switch` 했더니 파일이 사라짐	정상 동작 — 브랜치마다 파일이 다름	git switch 로 원래 브랜치로 돌아오면 다시 나타남

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- ☐ 브랜치를 만들면 커밋이 복사되나요?
- ☐ fast-forward 로 병합되는 조건은 무엇인가요?
- ☐ 병합 커밋의 부모는 몇 개인가요?
- ☐ 충돌 표시자를 지우지 않고 커밋하면 어떻게 되나요?
- ☐ 커밋하기 애매한 작업을 남겨둔 채 브랜치를 옮기려면 어떻게 하나요?

한 줄 정리

브랜치는 복사가 아니라 이름표입니다. 그래프의 모양이 병합 방식을 결정하고, 충돌은 같은 줄을 양쪽에서 고쳤을 때만 생깁니다.

다음 시간 → 3교시 — 내 컴퓨터를 벗어나 팀과 주고받기

WORKSHEET

막힌 STEP 번호, 화면에 나온 오류 메시지, 강사에게 물어볼 것을 적어 두세요.